“Linux Command Execution Time Profiler”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

D Sri Varshini	**2520080013**
Ch Karthikeya	**2520080022**
P Shreyas	**2520080001**

Under the Esteemed Guidance of

Raghupathi M
Assistant Professor
Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux Command Execution Time Profiler"** is a record of Bonafide work of **D Sri Varshini - 2520080013, Ch Karthikeya - 2520080022, P Shreyas – 2520080001** submitted in partial fulfillment for the award of B. Tech in Department Of Artificial Intelligence and Data Science to K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

D Sri Varshini	**2520080013**
Ch Karthikeya	**2520080022**
P Shreyas	**2520080001**

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is certify that the mini project based report entitled **"Linux Command Execution Time Profiler"** is a bonafide work done and submitted by **D Sri Varshini - 2520080013, Ch Karthikeya - 2520080022, P Shreyas - 2520080001,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Artificial Intelligence and Data Science, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr. K. Anusha, and my guide **Raghupathi M ,**Department of Computer Science and Information Technology which cannot be expressed in words for his tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

D Sri Varshini	**2520080013**
Ch Karthikeya	**2520080022**
P Shreyas	**2520080001**

Abstract

The Linux Command Execution Time Profiler is an operating systems project that executes a Linux command and measures its elapsed time and resource usage. Developers and students need to distinguish the total time a command takes from the CPU time it actually consumes. A waiting command may have a long elapsed duration while using very little CPU. This project provides a terminal-based tool for observing these differences through Linux process-management and timing interfaces.

The system is implemented in C and runs on Ubuntu Linux, including Ubuntu through Windows Subsystem for Linux (WSL). The program accepts a command and its arguments from the terminal. It uses fork() to create a child process and execvp() to replace the child with the requested command. The parent uses wait4() to wait for completion and obtain resource statistics. CLOCK_MONOTONIC with clock_gettime() measures elapsed time without depending on changes to the calendar clock.

After completion, the profiler displays the command name, child process ID, elapsed time, user CPU time, system CPU time, peak resident memory, and exit status. It also reports termination by a signal and handles invalid commands. Pipelines and shell built-ins can be executed through bash -c. The shell can redirect the output into a text file or use tee to display and save the same execution profile.

The project demonstrates process creation, program execution, parent-child synchronization, resource accounting, and interpretation of process termination status. It uses standard C headers and Linux interfaces with GCC as the compiler. The implementation provides a compact educational profiler rather than a full benchmarking framework. Measurements include process startup overhead, and repeated runs may vary with system load. The resulting tool supports practical comparison of waiting commands and CPU-intensive commands in a Linux terminal.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		7
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Objectives of the Project	
2	OS Concepts and Background Study		8
		2.1 Operating System Concepts Related to the Project	
3	Technologies Used		9
4	System Architecture and Design		10–11
		4.1 Command Line Interface	
		4.2 Profiling Engine	
		4.3 Resource Usage Report	
		4.4 Working Methodology	
		4.5 Implementation Modules	
5	Testing and Validation		12–13
		5.1 Screenshots of Test Execution	
6	Results		14
7	Limitations		15
8	Future Enhancements		16
9	Team Contributions		17
10	Conclusion		18

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Linux commands consume both execution time and system resources. Elapsed time includes CPU execution, scheduling, and waiting, whereas CPU time measures processing performed in user and kernel modes. Understanding this distinction helps explain command behaviour.

The profiler runs as a parent process and launches the requested command as its child. It waits for completion, collects resource statistics, and prints a terminal report. Ubuntu on WSL provides the Linux environment while Windows remains the host operating system.

1.2 Problem Statement

To develop a C program on Ubuntu that executes a user-specified Linux command and reports elapsed time, CPU usage time, peak resident memory, and termination status through the terminal.

1.3 Objectives of the Project

- Accept a command and its arguments from the terminal.
- Create a child process and execute the command.
- Measure elapsed time using a monotonic clock.
- Display the child process ID and termination status.
- Report user CPU time and system CPU time.
- Report peak resident memory and support shell output redirection.
- Demonstrate operating-system process-management concepts.

CHAPTER -2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

- **Process creation: fork() creates a child process.**
- **Program execution:** execvp() replaces the child process image and searches PATH for the command.
- **Process synchronization:** wait4() waits for the child and collects its status and resource usage.
- **Timing:** clock_gettime(CLOCK_MONOTONIC) measures elapsed time independently of calendar-clock changes.
- **Parent-child relationship:** The parent profiles execution while the child runs the requested command.
- **Termination handling:** WIFEXITED and WIFSIGNALED distinguish normal exit from signal termination.

CHAPTER -3 TECHNOLOGIES USED

- **Programming language:** C
- **User interface:** Ubuntu terminal
- **System interfaces:** fork(), execvp(), wait4(), clock_gettime()
- **Operating system:** Ubuntu Linux / Ubuntu on WSL
- **Compiler:** GCC
- **Text editor:** nano
- **Version control:** Git and GitHub

CHAPTER -4 SYSTEM ARCHITECTURE AND DESIGN

The system consists of three main components.

4.1 Command Line Interface

The user invokes ./profiler followed by a command and its arguments. The interface checks that a command was supplied and displays usage instructions otherwise. The command output appears before the execution profile.

```
shreyas@Grigori:~$ mkdir -p ~/command-profiler
shreyas@Grigori:~$ cd ~/command-profiler
shreyas@Grigori:~/command-profiler$ nano profiler.c
shreyas@Grigori:~/command-profiler$ gcc -Wall -Wextra -O2 profiler.c -o profiler
shreyas@Grigori:~/command-profiler$ ls
profiler  profiler.c
shreyas@Grigori:~/command-profiler$ ./profiler command arguments
```

Figure 4.1: Command line interface and profile format

4.2 Profiling Engine

The C program records a start timestamp and creates a child using fork(). The child executes the requested command through execvp(). The parent calls wait4(), retries if interrupted by EINTR, and records the end timestamp after the child completes.

4.3 Resource Usage Report

The report converts monotonic timestamps and resource-usage values into seconds. It displays elapsed time, user and system CPU time, peak resident memory in KB, process ID, and exit status. Execution failures print an error and produce status 127.

4.4 Working Methodology

1. The user opens the Ubuntu terminal.
2. The user compiles profiler.c using GCC.
3. The user runs ./profiler with a command and arguments.
4. The profiler records the start time and creates a child.
5. The child replaces itself with the requested command.
6. The parent waits for completion and collects resource usage.
7. The profiler records the end time and calculates elapsed and CPU times.

8. The terminal displays the profile and termination status.
9. The user may save output using redirection or tee.

4.5 Implementation Modules

- Command-line argument validation
- Child process creation and command execution
- Monotonic timing and resource collection
- Terminal reporting and shell output capture
- Error handling and termination-status interpretation

CHAPTER -5 TESTING AND VALIDATION

The following test cases validate timing, command execution, error reporting, and output capture. Expected outcomes are specified without presenting unrecorded runs as measured results.

Test	Expected result
Delay command	sleep 2: elapsed time is approximately 2 seconds; CPU time is small.
File listing	ls -l: directory listing appears before the execution profile.
CPU-intensive command	A shell loop completes and reports its CPU and elapsed time.
Invalid command	An error is printed and the child exit status is 127.
Shell pipeline	bash -c runs the pipeline and the profiler reports completion.
Output capture	tee writes the displayed profile into a text file.

5.1 Screenshots of Test Execution

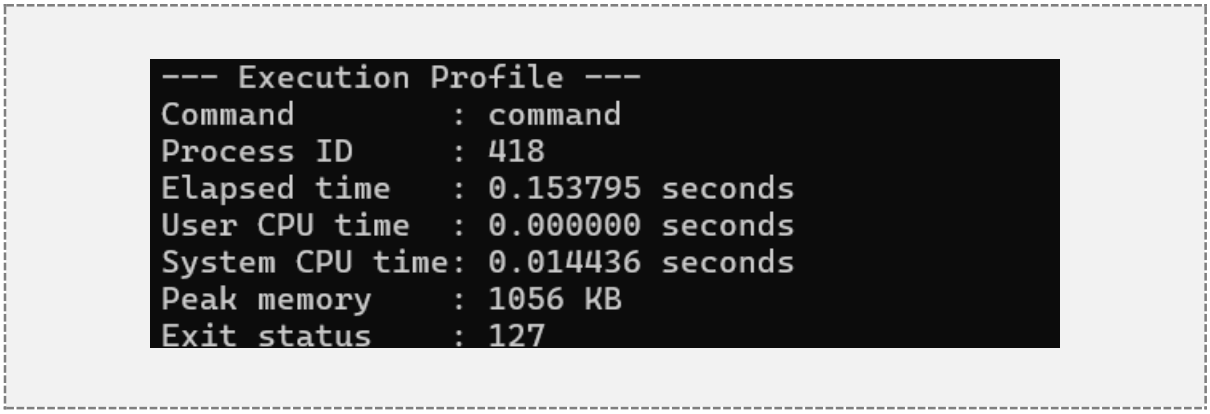


Figure 5.1

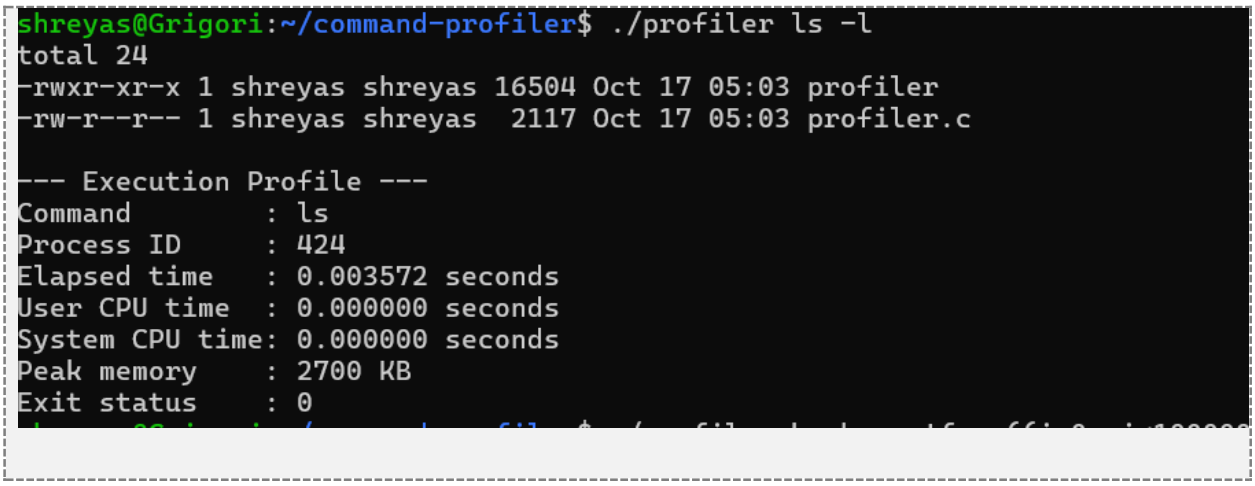


Figure 5.2

```
shreyas@Grigori:~/command-profiler$ ./profiler ls -l
total 24
-rwxr-xr-x 1 shreyas shreyas 16504 Oct 17 05:03 profiler
-rw-r--r-- 1 shreyas shreyas  2117 Oct 17 05:03 profiler.c

--- Execution Profile ---
Command         : ls
Process ID      : 424
Elapsed time    : 0.003572 seconds
User CPU time   : 0.000000 seconds
System CPU time: 0.000000 seconds
Peak memory     : 2700 KB
Exit status     : 0
```

Figure 5.3

```
Exit status     : 0
shreyas@Grigori:~/command-profiler$ ./profiler invalidcommand
invalidcommand: No such file or directory

--- Execution Profile ---
Command         : invalidcommand
Process ID      : 432
Elapsed time    : 0.168280 seconds
User CPU time   : 0.000000 seconds
System CPU time: 0.014311 seconds
Peak memory     : 1184 KB
Exit status     : 127
```

Figure 5.4

```
shreyas@Grigori:~/command-profiler$ ./profiler bash -c 'ls -l | wc -l'
3

--- Execution Profile ---
Command         : bash
Process ID      : 440
Elapsed time    : 0.006692 seconds
User CPU time   : 0.005164 seconds
System CPU time: 0.003212 seconds
Peak memory     : 3448 KB
Exit status     : 0
```

Figure 5.5

CHAPTER -6 RESULTS

The C implementation compiled with GCC using -Wall -Wextra -O2. A controlled sleep 0.1 test produced approximately 0.101 seconds of elapsed time, demonstrating measurement of a waiting command. The report also displayed CPU time, memory, process ID, and exit status. Exact values depend on the execution environment. The additional cases in Chapter 5 provide a repeatable validation procedure for the Ubuntu installation.

```
shreyas@Grigori:~/command-profiler$ cat result.txt

--- Execution Profile ---
Command         : sleep
Process ID      : 446
Elapsed time    : 2.057367 seconds
User CPU time   : 0.001699 seconds
System CPU time: 0.000000 seconds
Peak memory     : 1976 KB
Exit status     : 0
```

Figure 6.1

```
shreyas@Grigori:~/profiler-submission$ /tmp/profiler-demo sleep 2 | tee "Hackathon./sample_output.txt"

--- Execution Profile ---
Command         : sleep
Process ID      : 973
Elapsed time    : 2.045524 seconds
User CPU time   : 0.002363 seconds
System CPU time: 0.000000 seconds
Peak memory     : 1972 KB
Exit status     : 0
```

Figure 6.2

CHAPTER -7 LIMITATIONS

- The implementation relies on Linux interfaces, including wait4(), and requires Ubuntu or another compatible Linux environment.
- Timing includes fork, command startup, scheduling, and waiting overhead.
- The prototype profiles one command invocation at a time and does not calculate repeated-run statistics.
- Peak memory for a shell pipeline is not the sum of the simultaneous memory usage of all its processes.

CHAPTER -8 FUTURE ENHANCEMENTS

- Add repeated runs with minimum, maximum, and average execution times.
- Export structured reports in CSV or JSON format.
- Compare multiple commands and their resource usage.
- Add optional timestamps and command history.
- Provide a graphical interface for viewing saved results.
- Add configurable timeouts and further automated tests.

CHAPTER -9 TEAM CONTRIBUTIONS

- **D Sri Varshini – 2520080013:** Implemented command-line argument validation, child process creation using `fork()`, and command execution using `execvp()`.
- **Ch Karthikeya – 2520080022:** Implemented elapsed-time measurement using `clock_gettime()`, resource collection using `wait4()`, and execution-profile reporting.
- **P Shreyas – 2520080001:** Handled integration, compilation and testing on Ubuntu, error-case validation, GitHub submission, documentation, and presentation preparation.

CHAPTER -10 CONCLUSION

The Linux Command Execution Time Profiler demonstrates how Linux process-management and resource-accounting interfaces can measure command execution. The C program combines fork(), execvp(), wait4(), and a monotonic clock to report elapsed time, CPU time, peak memory, and termination status. Its terminal workflow supports practical learning and provides a foundation for repeated benchmarking and structured reporting.